

비동기 I – 이벤트 루프와 Promise



타이머 함수

setTimeout

setInterval

자바스크립트에는 특수한 역할을 하는 함수가 두 개 존재한다.



타이머 함수

setTimeout

setInterval

즉시 실행되는 일반 함수 호출과는 달리,
위 두 함수를 사용하면 **함수 호출을 예약**할 수 있다.



setTimeout

```
console.log('3초 후에 메시지가 출력됩니다.');
```

```
setTimeout(() => {  
  console.log('3초가 지났습니다!');  
}, 3000);
```

‘setTimeout’ 함수를 사용하면 **특정 시간 후에 콜백 함수가 호출**되도록 예약할 수 있다.

두 번째 인수로 시간을 전달하며, 단위는 밀리초(ms)이다.



setTimeout

```
setTimeout((name) => console.log(`Hello, ${name}!`), 2000, "Elice") // 2초 후에 'Hello, Elice!' 출력
```

세 번째 이후의 인수에는 콜백 함수에 전달될 인수를 전달할 수 있다.


```
const timeoutId = setTimeout(() => console.log(`Hello!`), 2000);  
  
console.log(timeoutId); // 1  
clearTimeout(timeoutId);
```

‘setTimeout’ 함수는 숫자 형태의 timeout id를 반환한다. (브라우저 환경 기준)
‘clearTimeout’ 함수에 이 id를 인수로 넘겨 호출하면 해당 timeout을 제거할 수 있다.

```
let count = 0;

const intervalId = setInterval(() => {
  count++;
  console.log(`Interval 실행: ${count}초 경과`);
}, 1000); // 1초마다 실행
```

```
'Interval 실행: 1초 경과'
'Interval 실행: 2초 경과'
'Interval 실행: 3초 경과'
'Interval 실행: 4초 경과'
'Interval 실행: 5초 경과'
'Interval 실행: 6초 경과'
'Interval 실행: 7초 경과'
'Interval 실행: 8초 경과'
...
```

‘setInterval’ 함수를 사용하면 **특정 시간마다 콜백 함수가 호출**되도록 예약할 수 있다.
두 번째 인수로 시간을 전달하며, 단위는 밀리초(ms)이다.

```
let count = 0;

const intervalId = setInterval(() => {
  count++;
  console.log(`Interval 실행: ${count}초 경과`);

  if (count === 5) {
    clearInterval(intervalId); // 5초 후에 반복 중지
    console.log('Interval 중지');
  }
}, 1000); // 1초마다 실행
```

```
'Interval 실행: 1초 경과'
'Interval 실행: 2초 경과'
'Interval 실행: 3초 경과'
'Interval 실행: 4초 경과'
'Interval 실행: 5초 경과'
'Interval 중지'
```

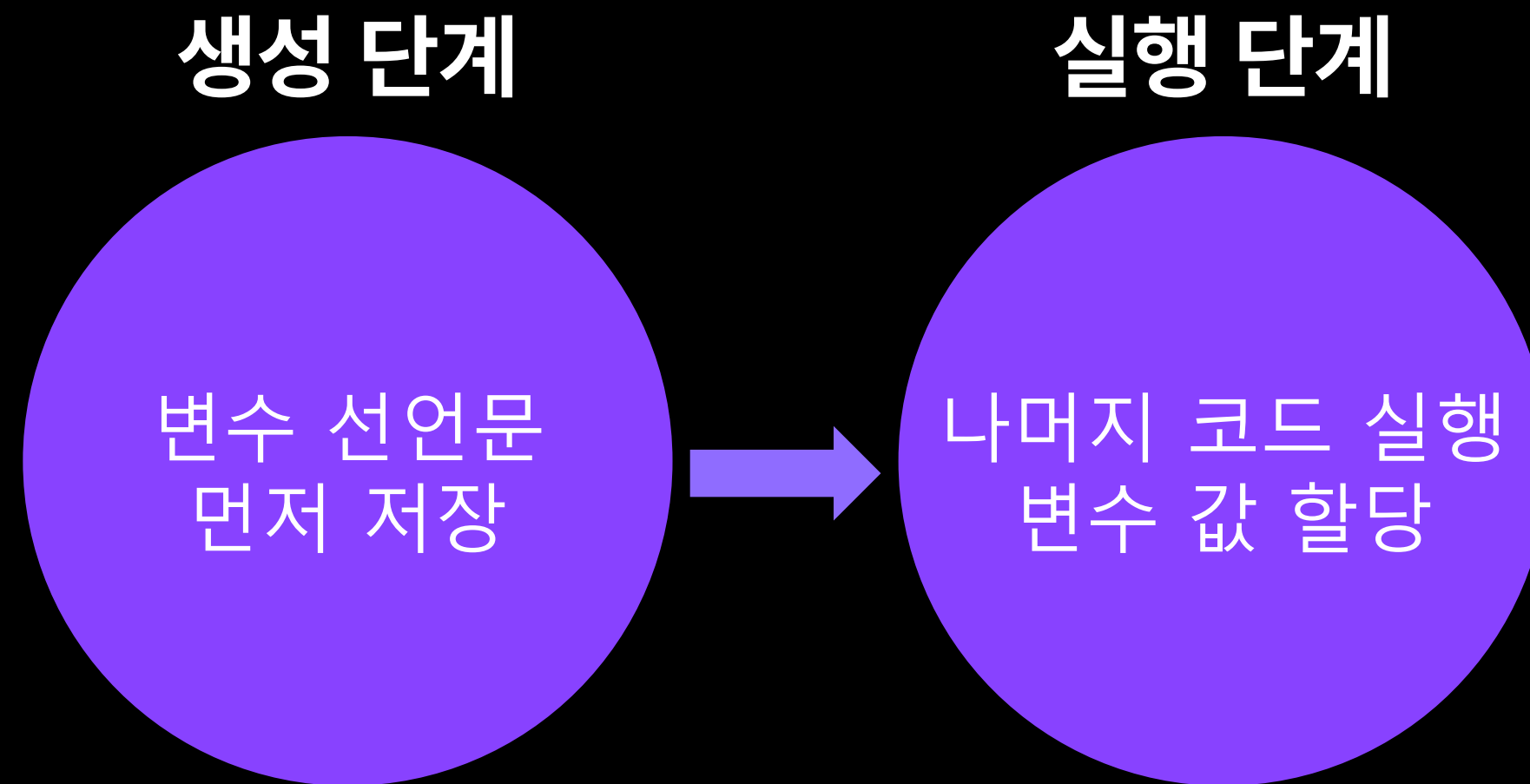
‘setInterval’ 함수는 숫자 형태의 interval id를 반환한다. (브라우저 환경 기준)

‘clearInterval’ 함수에 이 id를 인수로 넘겨 호출하면 **해당 interval을 제거**할 수 있다.



작성한 자바스크립트 코드는 자바스크립트 엔진을 통해 파싱되고 실행된다.

가장 많이 사용하는 Chrome 브라우저의 경우 V8 엔진을 사용한다.



자바스크립트 엔진은 코드 실행 전 **실행 컨텍스트***라는 것을 생성한다.
실행 컨텍스트는 위의 두 단계를 통해 생성 및 사용된다.



실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

전역 실행 컨텍스트 생성

그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.



실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

func1 실행 컨텍스트 생성



그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.



실행 컨텍스트

```
const x = 10;

function func1() {
  const x = 20;

  func2();
}

function func2() {
  console.log(x);
}

func1();
func2();
```

func2 실행 컨텍스트 생성



그리고 실행 컨텍스트는 각 스코프가 실행되기 직전에 생성된다.



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

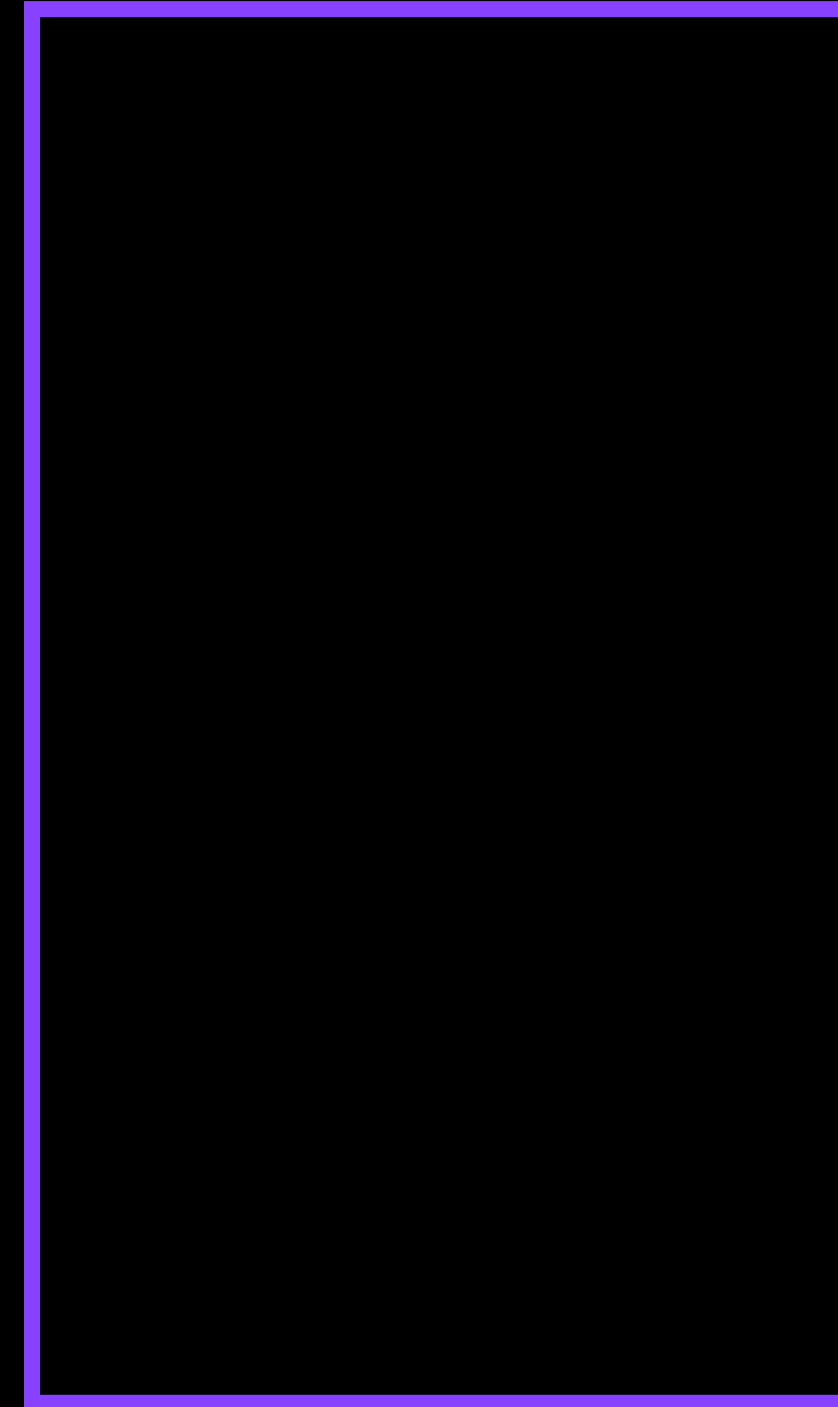
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```



생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 outerFunc
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 innerFunc
실행 컨텍스트

함수 outerFunc
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

함수 outerFunc
실행 컨텍스트

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

전역 실행 컨텍스트

생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



실행 컨텍스트 스택

```
const global = 10;

function outerFunc() {
  const global = 20;
  const local1 = 30;

  function innerFunc() {
    const local1 = 40;
    const local2 = 50;

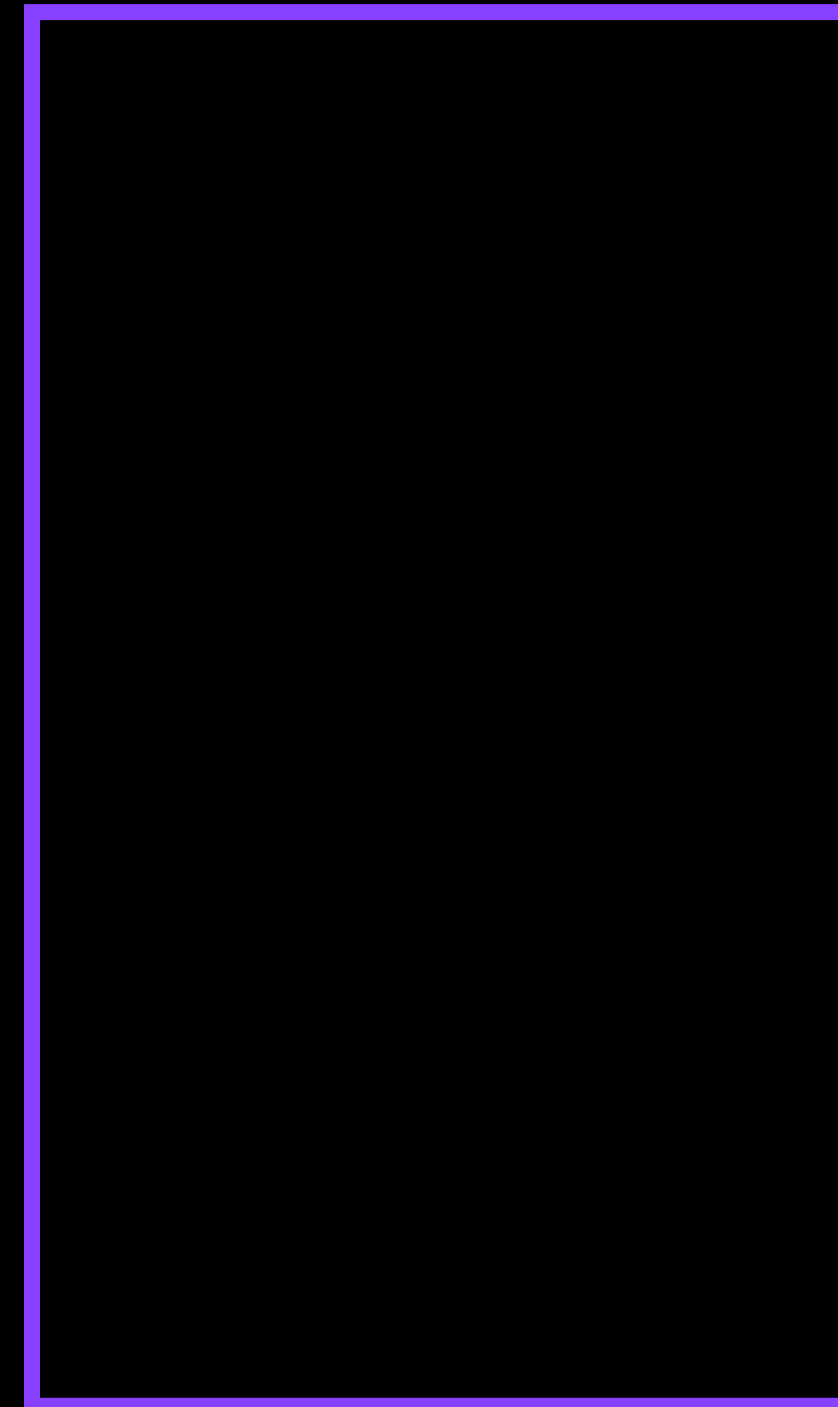
    console.log("inner function");
    console.log(global);
    console.log(local1);
    console.log(local2);
  }

  console.log("outer function");
  console.log(global);
  console.log(local1);

  innerFunc();
}

console.log("global");
console.log(global);

outerFunc();
```

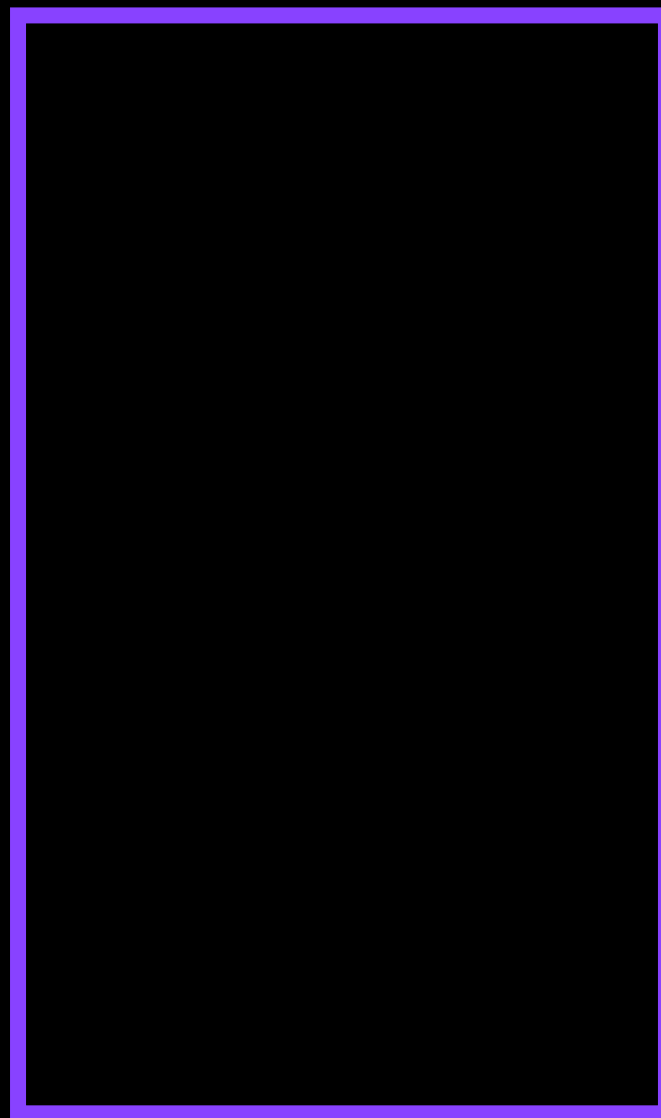


생성되는 실행 컨텍스트는 모두 **실행 컨텍스트 스택(콜 스택)**에서 관리된다.

스택(Stack): 후입선출(LIFO, Last In First Out) 원칙에 따라 데이터를 저장하고 제거하는 선형 자료구조



싱글 스레드 (Single Thread)

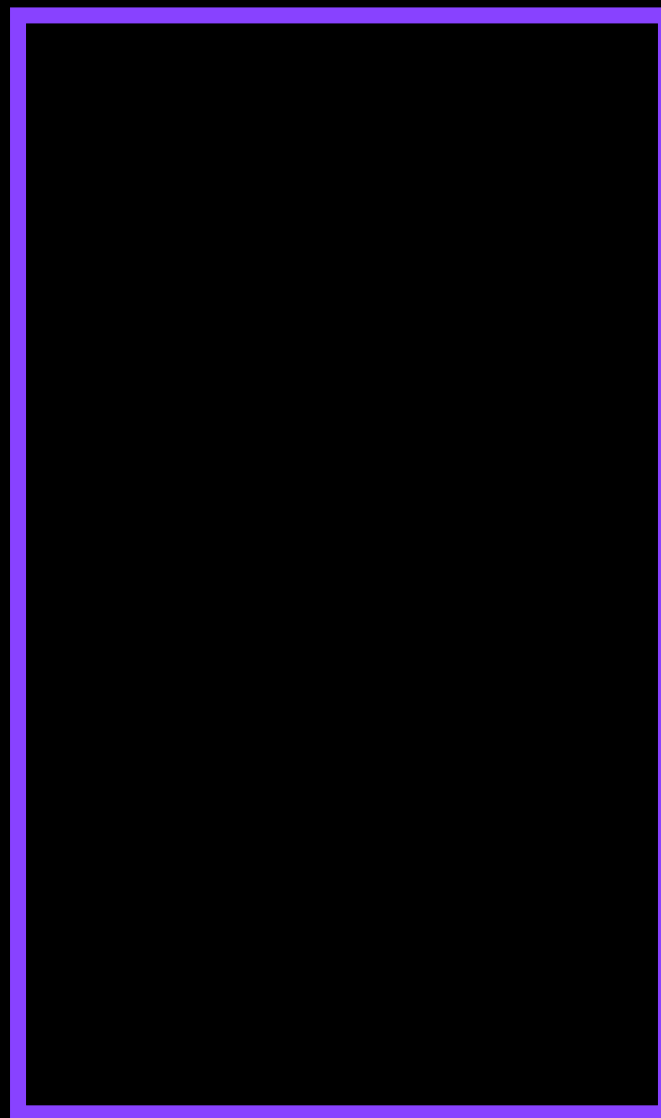


자바스크립트는 오직 하나의 실행 컨텍스트 스택(콜 스택)을 가진다.

즉, **한 번에 하나의 작업만 수행**할 수 있다.



싱글 스레드 (Single Thread)



= 자바스크립트는 **싱글 스레드** 기반 언어이다.



싱글 스레드의 문제점

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 동기적인 파일 읽기
const data = fs.readFileSync('largeFile.txt', 'utf8');

console.log('파일 읽기 완료');
console.log(data);

console.log('다른 작업 실행');
```

싱글 스레드에서는 한 번에 하나의 작업만 수행하므로 **작업 순서가 보장**된다는 장점이 있지만,
앞선 작업이 종료되기 전까지 이후 작업을 시작하지 못한다는 문제가 있다.



싱글 스레드의 문제점

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 동기적인 파일 읽기
const data = fs.readFileSync('largeFile.txt', 'utf8');

console.log('파일 읽기 완료');
console.log(data);

console.log('다른 작업 실행');
```

앞선 작업이 종료되기 전까지 이후 작업을 시작하지 못한다는 문제가 있다.

= **블로킹(Blocking)**이 발생한다.



싱글 스레드의 문제점

```
const fs = require('fs');  
  
console.log('파일 읽기 시작');  
  
// 동기적인 파일 읽기  
const data = fs.readFileSync('largeFile.txt', 'utf8');  
  
console.log('파일 읽기 완료');  
console.log(data);  
  
console.log('다른 작업 실행');
```



```
fs.readFileSync('largeFile.txt', 'utf8');
```

앞선 작업이 종료되기 전까지 이후 작업을 시작하지 못한다는 문제가 있다.

= **블로킹(Blocking)**이 발생한다.



동기 (Synchronous) 처리 방식

```
const fs = require('fs');  
  
console.log('파일 읽기 시작');  
  
// 동기적인 파일 읽기  
const data = fs.readFileSync('largeFile.txt', 'utf8');  
  
console.log('파일 읽기 완료');  
console.log(data);  
  
console.log('다른 작업 실행');
```



```
fs.readFileSync('largeFile.txt', 'utf8');
```

동기 (Synchronous) 처리 방식

작업이 순차적으로 실행되며, 현재 작업이 완료될 때까지 다음 작업이 대기하는 방식

비동기 (Asynchronous) 처리 방식

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 비동기적인 파일 읽기
fs.readFile('largeFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('파일 읽기 완료');
  console.log(data);
});

console.log('다른 작업 실행');
```

비동기 (Asynchronous) 처리 방식

작업이 시작된 후 완료 여부와 상관없이 다음 작업을 즉시 실행하며,
작업 완료 시 콜백 함수나 프로미스 등을 통해 결과를 처리하는 방식

비동기 (Asynchronous) 처리 방식

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 비동기적인 파일 읽기
fs.readFile('largeFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('파일 읽기 완료');
  console.log(data);
});

console.log('다른 작업 실행');
```

```
fs.readFile
('largeFile.txt', 'utf8',
() => { ... });
```

비동기 (Asynchronous) 처리 방식

작업이 시작된 후 완료 여부와 상관없이 다음 작업을 즉시 실행하며,
작업 완료 시 콜백 함수나 프로미스 등을 통해 결과를 처리하는 방식

비동기 (Asynchronous) 처리 방식

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 비동기적인 파일 읽기
fs.readFile('largeFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('파일 읽기 완료');
  console.log(data);
});

console.log('다른 작업 실행');
```

```
console.log('다른  
작업 실행');
```

비동기 (Asynchronous) 처리 방식

작업이 시작된 후 완료 여부와 상관없이 다음 작업을 즉시 실행하며,
작업 완료 시 콜백 함수나 프로미스 등을 통해 결과를 처리하는 방식

비동기 (Asynchronous) 처리 방식

```
const fs = require('fs');

console.log('파일 읽기 시작');

// 비동기적인 파일 읽기
fs.readFile('largeFile.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log('파일 읽기 완료');
  console.log(data);
});

console.log('다른 작업 실행');
```

파일 로드 완료 후

(err, data) => { ... }

비동기 (Asynchronous) 처리 방식

작업이 시작된 후 완료 여부와 상관없이 다음 작업을 즉시 실행하며,
작업 완료 시 콜백 함수나 프로미스 등을 통해 결과를 처리하는 방식

비동기 (Asynchronous) 처리 방식 종류

타이머 함수

(setTimeout / setInterval)

Ajax

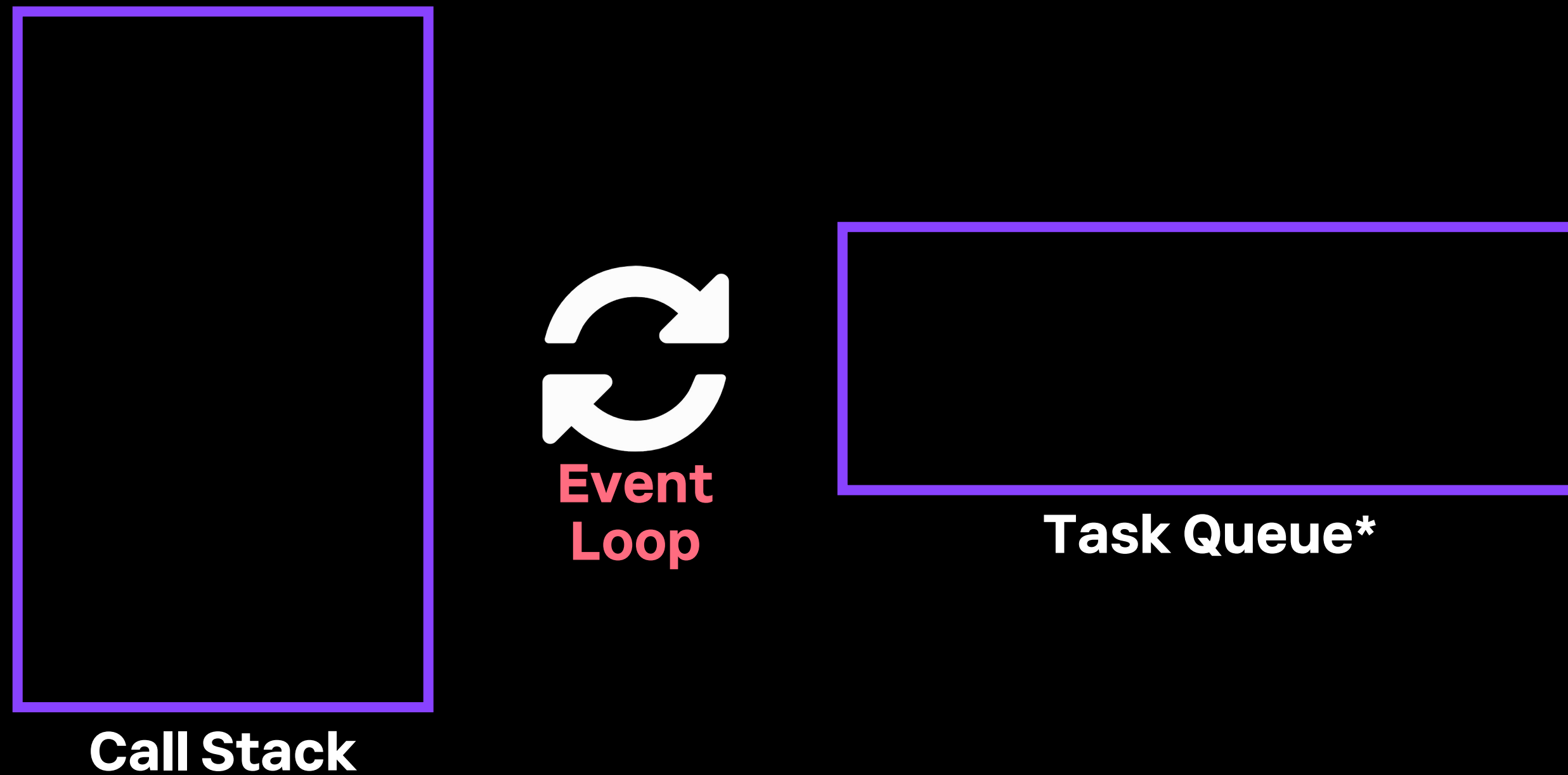
(XMLHttpRequest, Fetch API 등)

이벤트 핸들러

(addEventListener)

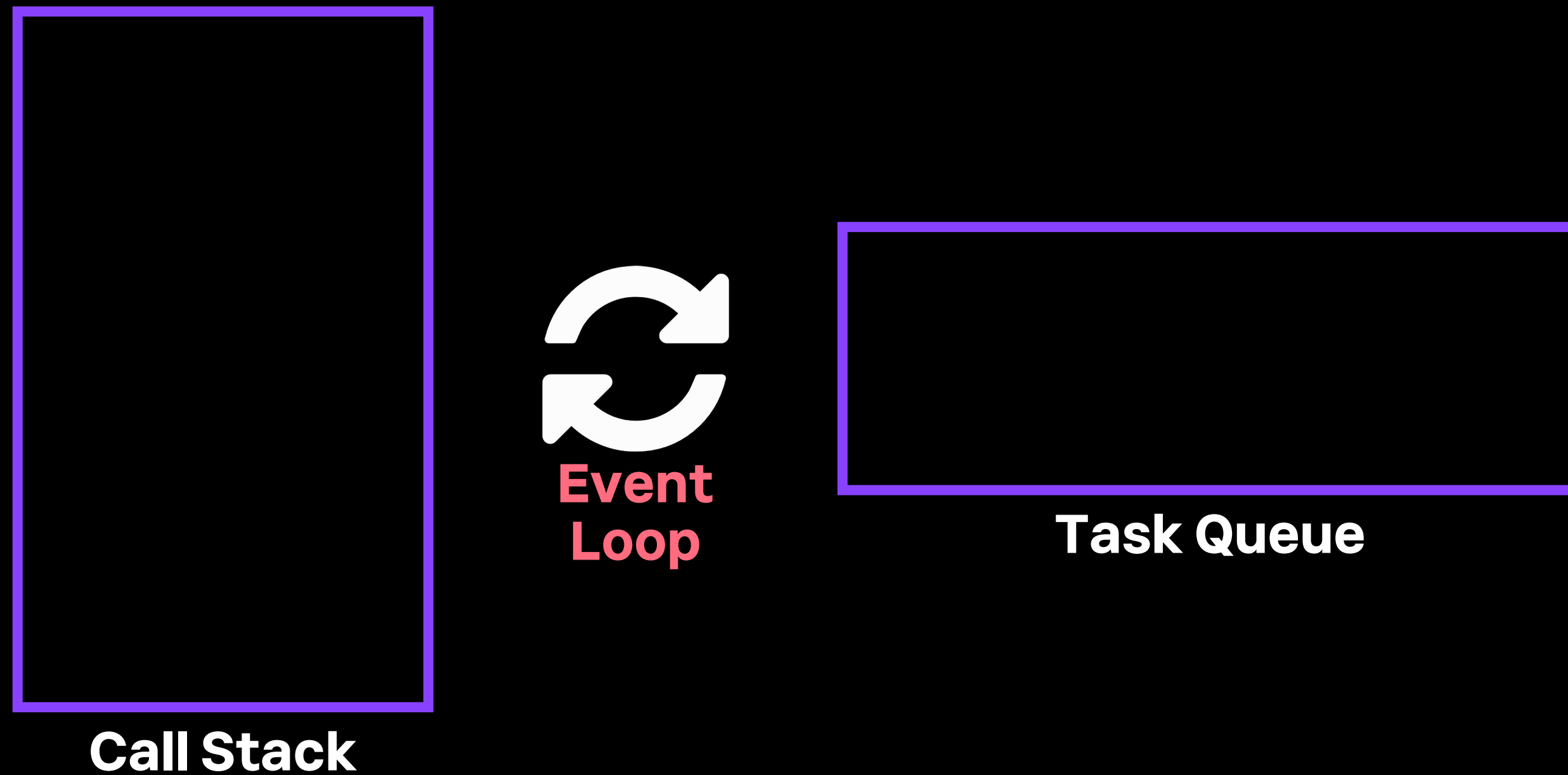
비동기 처리 방식으로 동작하는 자바스크립트 기능은
위 3가지이다.

비동기 (Asynchronous) 동작 원리



브라우저는 이벤트 큐 (Event Queue)와 태스크 큐 (Task Queue)를 이용해
비동기를 처리한다.

비동기 (Asynchronous) 동작 원리



이벤트 루프 (Event Loop): 콜 스택이 비어 있고, 태스크 큐에 함수가 존재하면 이를 콜 스택으로 이동시킴

태스크 큐* (Task Queue): 이벤트 핸들러 및 비동기 함수의 콜백 함수가 잠시 저장되는 큐 영역



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

위 코드를 동기 처리 방식으로 실행하면,
1 → 3 → 2 순서로 출력되는 것이 정상이다.

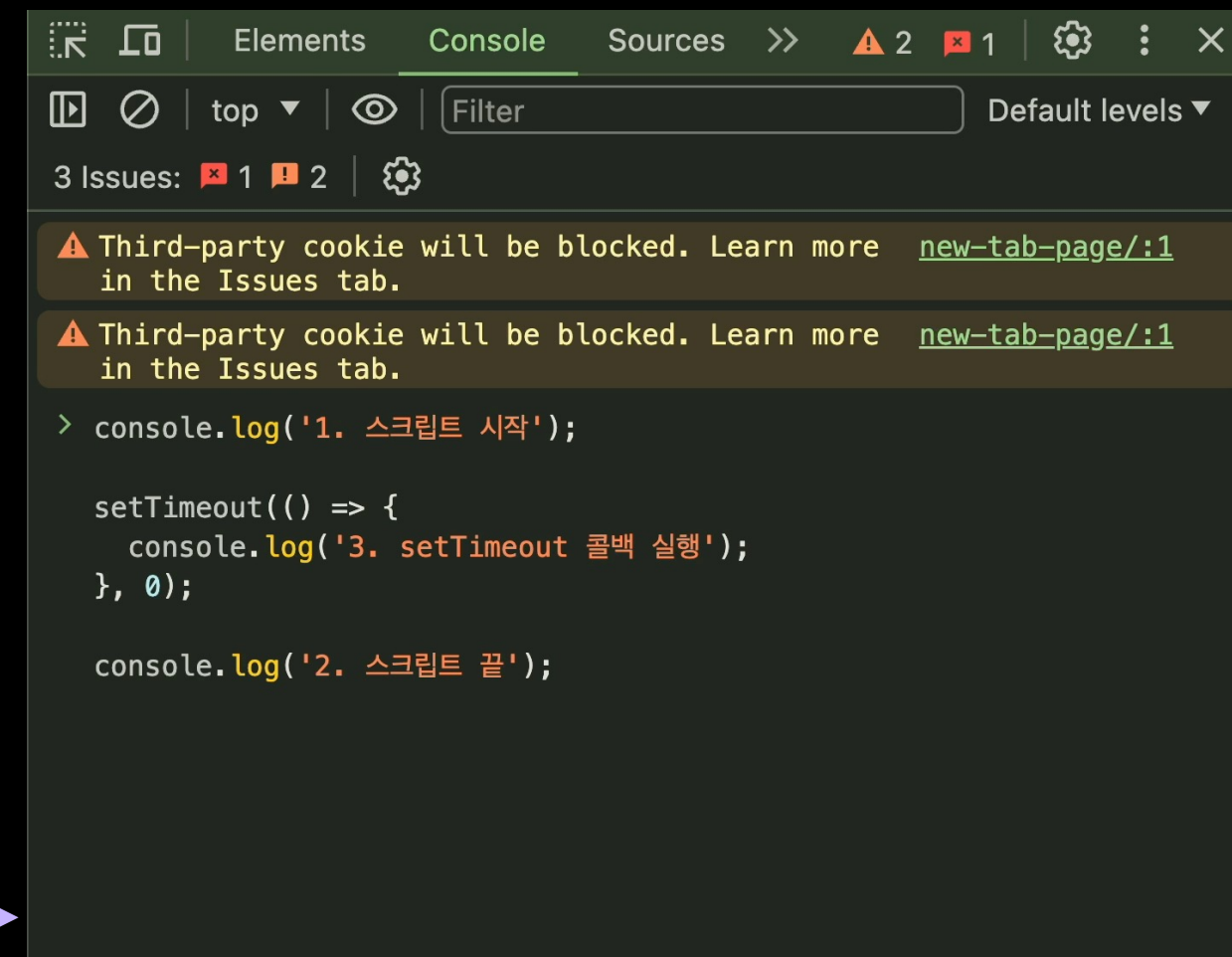
비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

Play →



그러나 실제로는 1 → 2 → 3 순으로 출력된다.

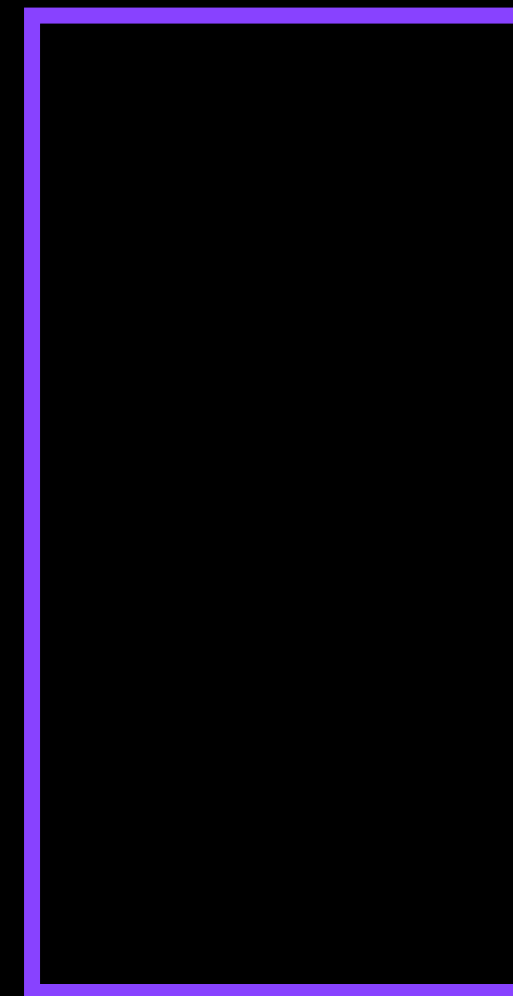
비동기의 동작 원리를 이해해 보면서 이유를 확인해보자.

비동기 (Asynchronous) 동작 원리

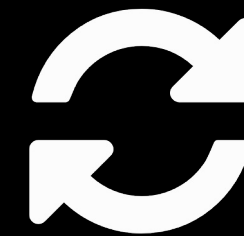
```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```



Call Stack



Event
Loop



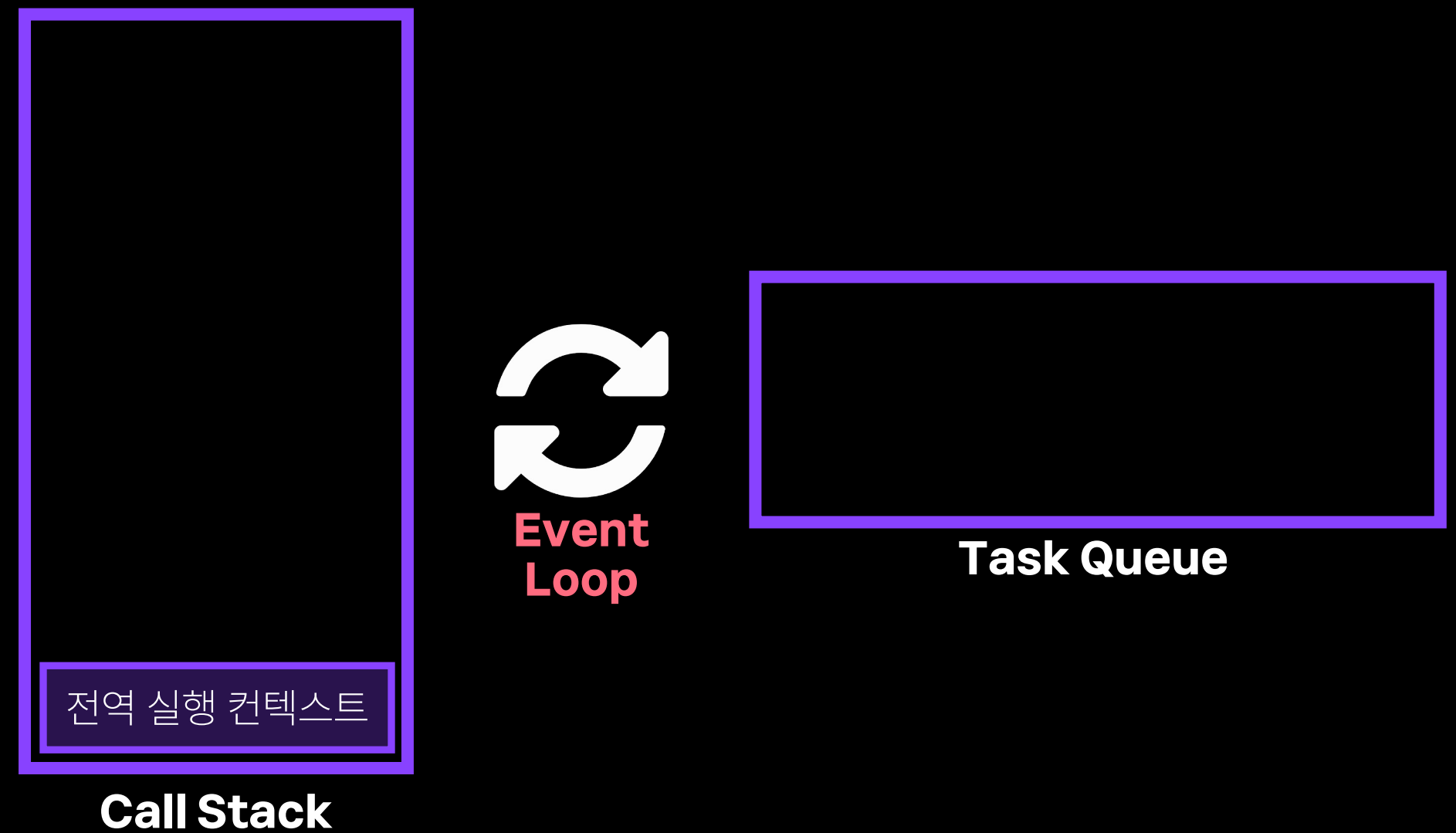
Task Queue

비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

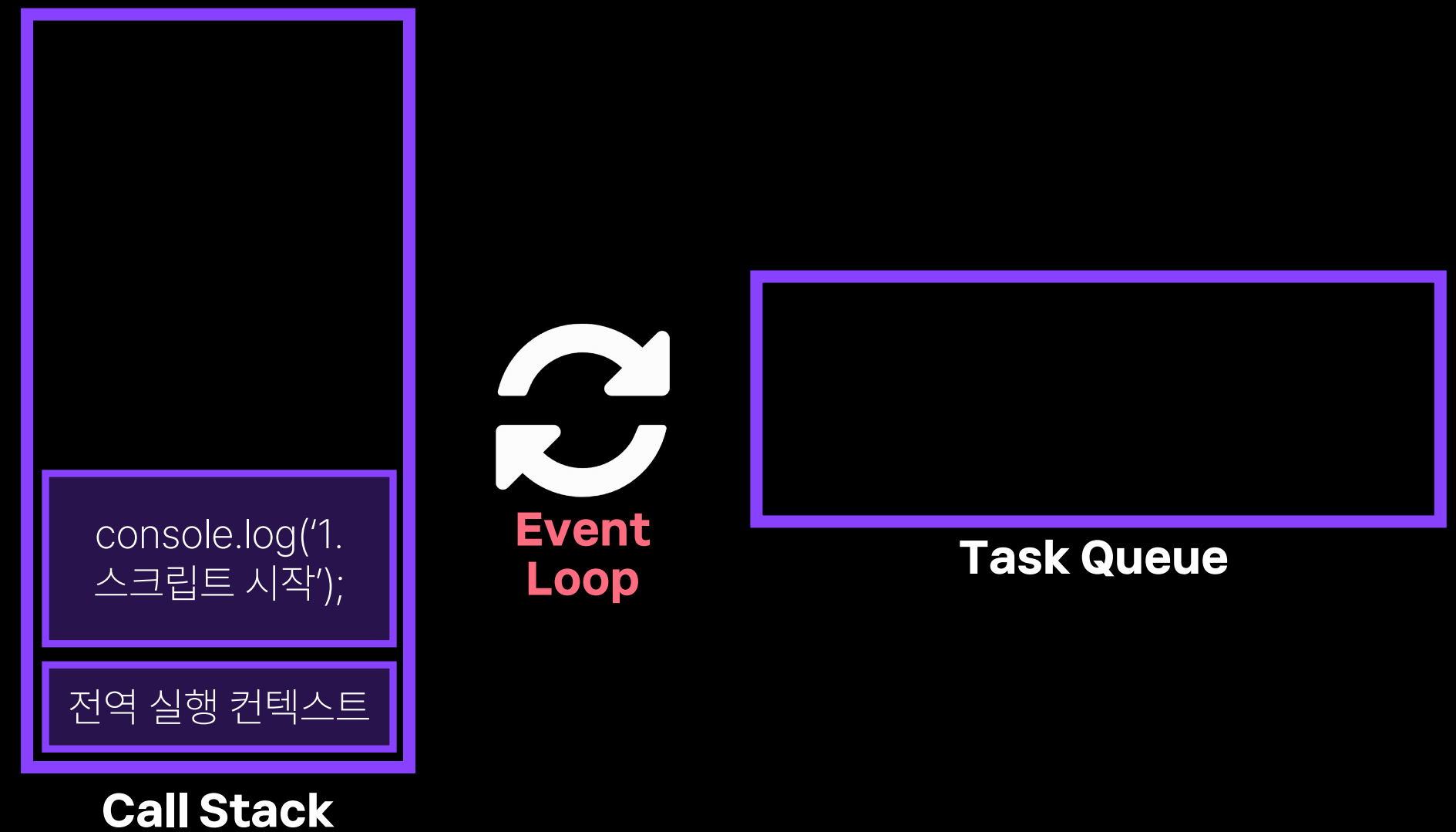


비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

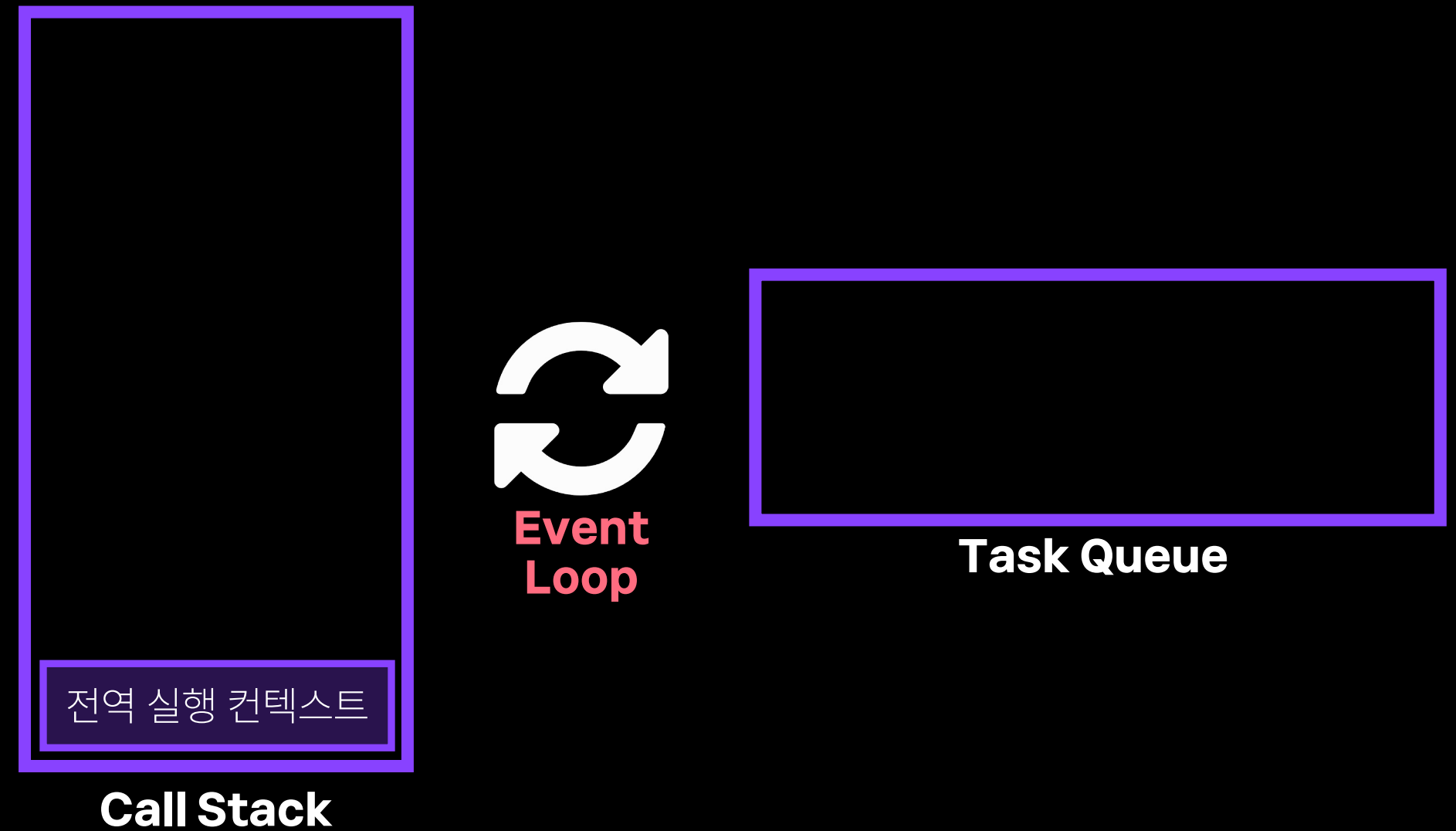
console.log('2. 스크립트 끝');
```



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

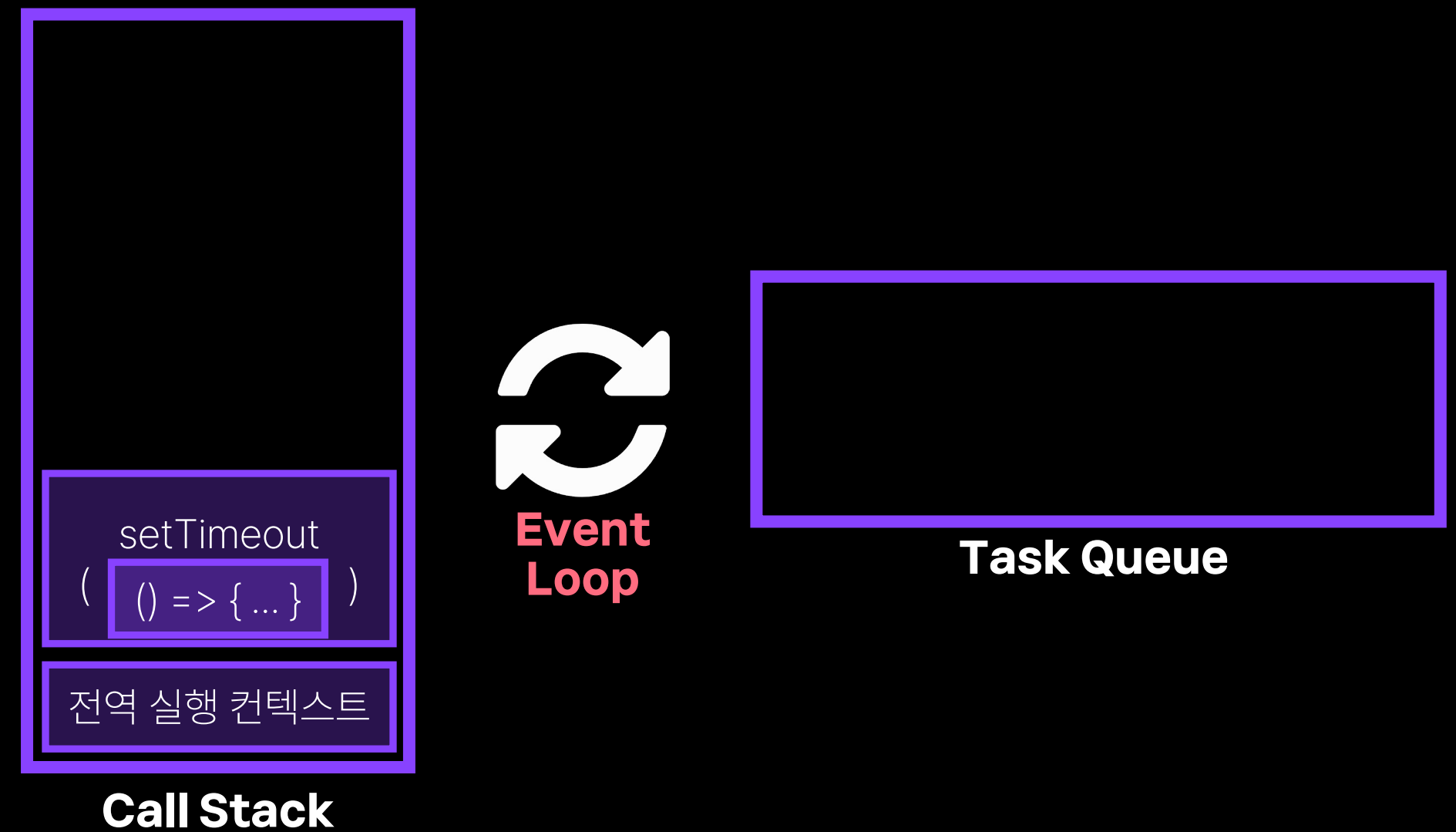
1. 스크립트 시작



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

1. 스크립트 시작



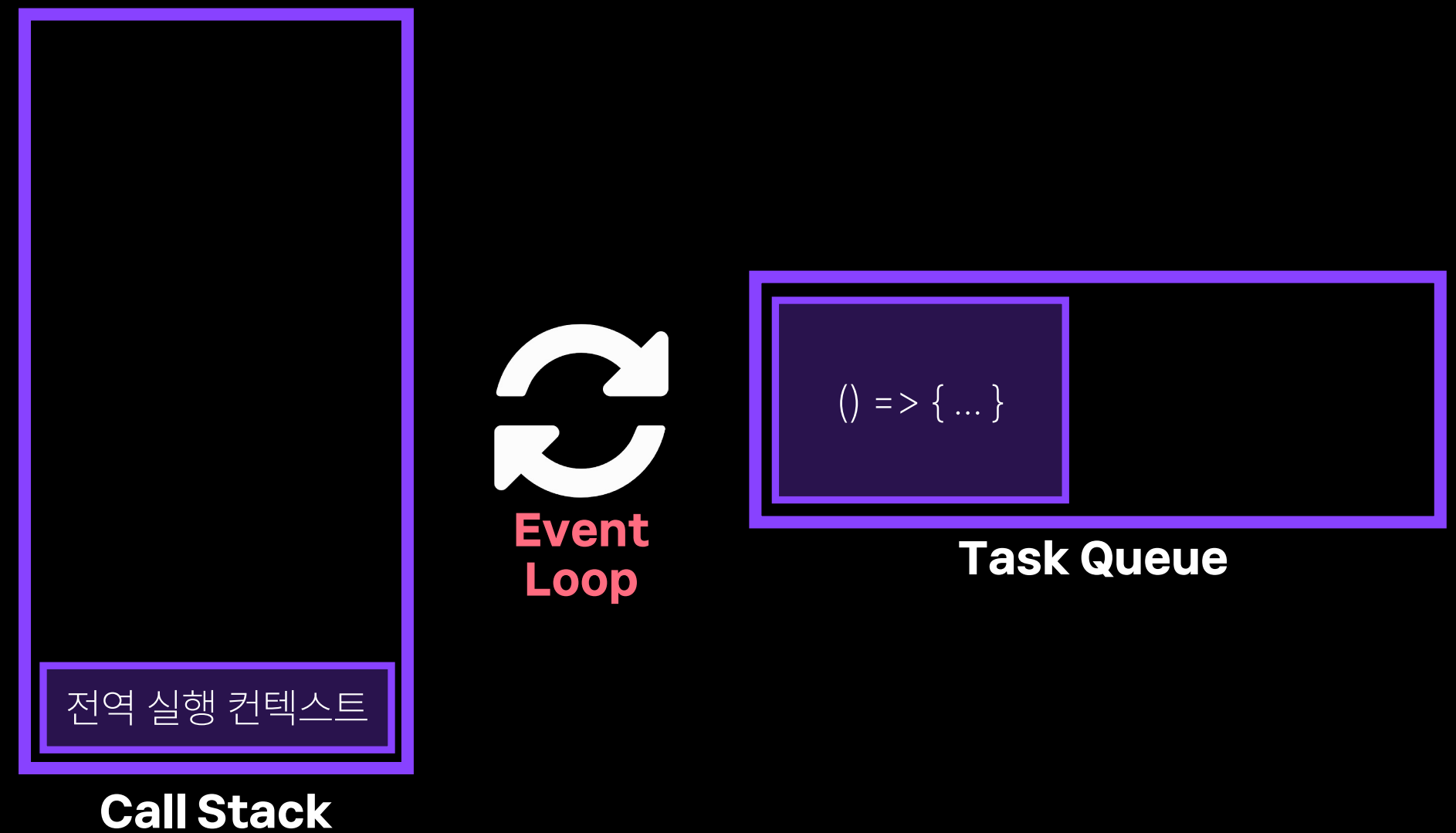
비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

1. 스크립트 시작



이벤트 루프가 비동기 함수의 콜백 함수를 태스크 큐에 저장한다.

(setTimeout의 경우, 지정된 밀리초가 지난 후에 태스크 큐에 저장된다.)

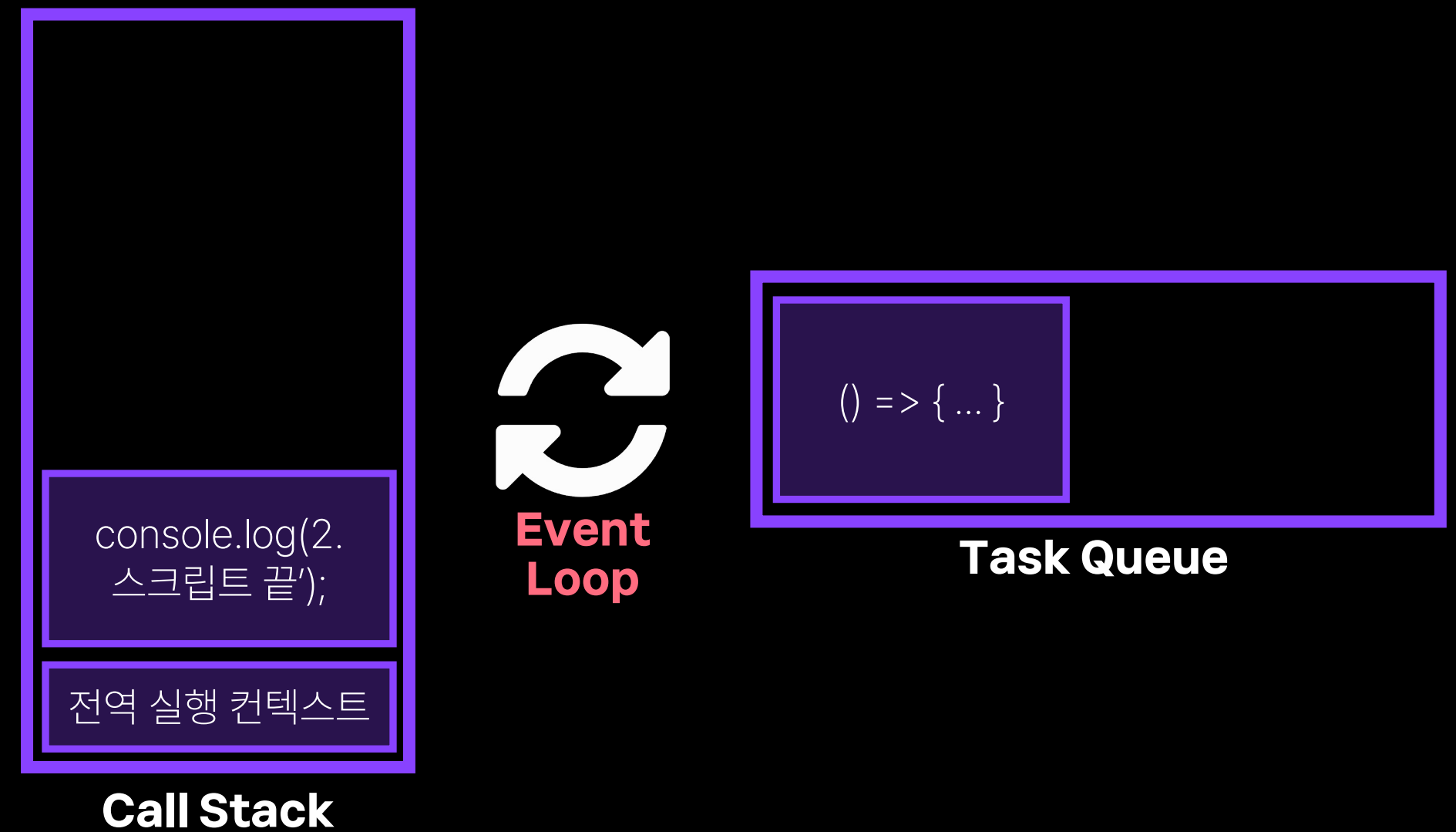
비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

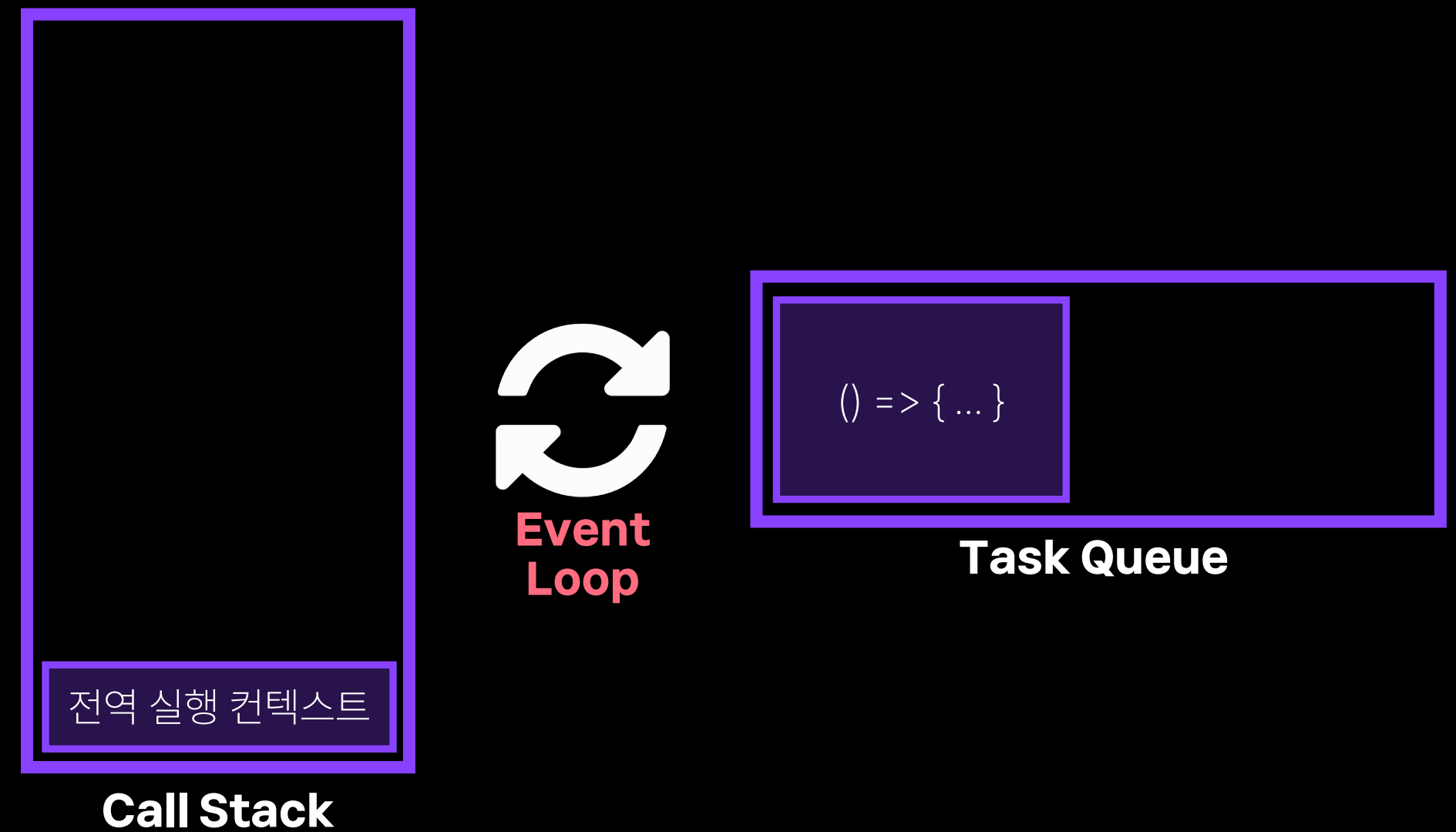
1. 스크립트 시작



비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

1. 스크립트 시작
2. 스크립트 끝



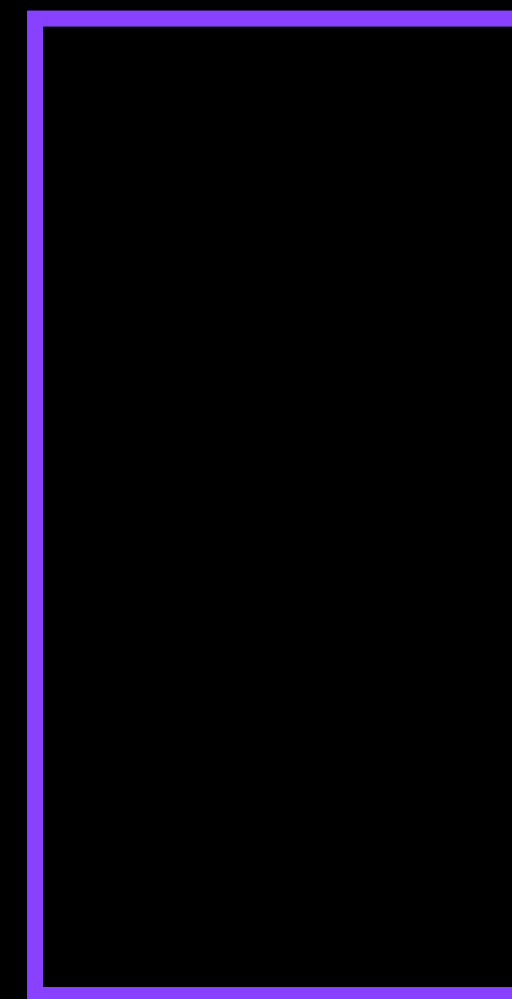
비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');

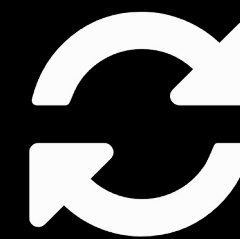
setTimeout(() => {
  console.log('3. setTimeout 콜백 실행');
}, 0);

console.log('2. 스크립트 끝');
```

1. 스크립트 시작
2. 스크립트 끝



Call Stack



Event Loop

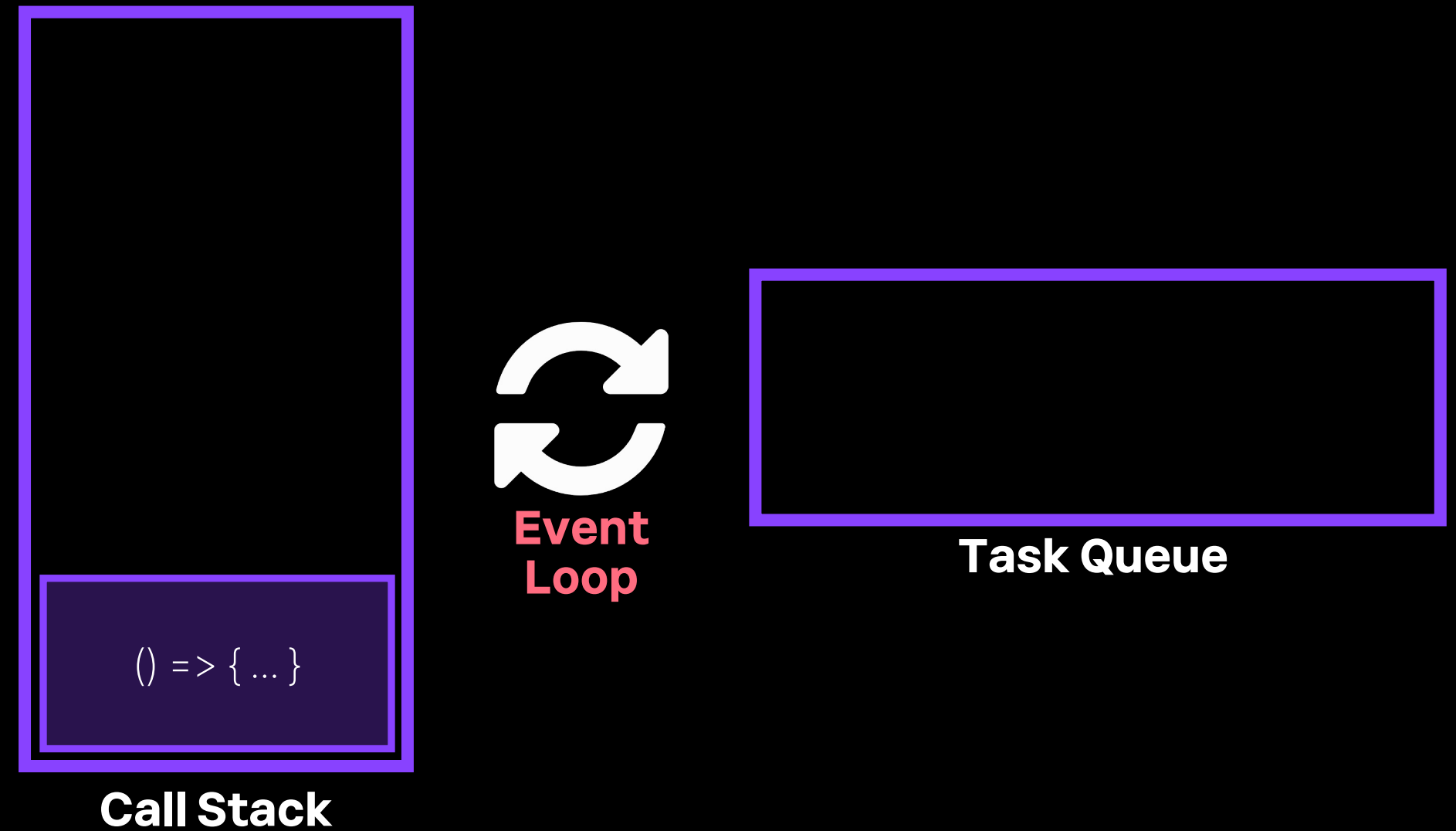


Task Queue

비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

1. 스크립트 시작
2. 스크립트 끝

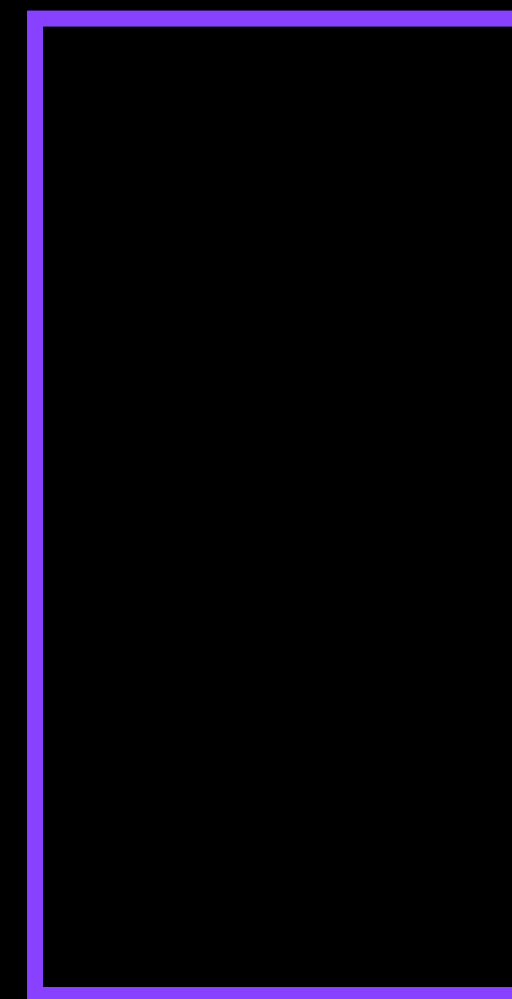


콜 스택이 비면, 이벤트 루프는 태스크 큐에 있는 작업을 콜 스택으로 이동시킨다.

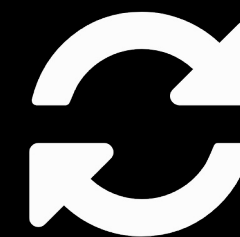
비동기 (Asynchronous) 동작 원리

```
console.log('1. 스크립트 시작');  
  
setTimeout(() => {  
  console.log('3. setTimeout 콜백 실행');  
}, 0);  
  
console.log('2. 스크립트 끝');
```

1. 스크립트 시작
2. 스크립트 끝
3. setTimeout 콜백 실행



Call Stack



Event
Loop



Task Queue



Promise

```
let name = "Elice";

setTimeout(() => {
  name = "Hellobit";
}, 2000);

console.log(name); // 'Elice'
```

기본적인 비동기 처리는 처리 결과를 활용할 수 없다.

(콜 스택이 빈 후에 콜백 함수가 호출되기 때문)



Promise

```
try {
  setTimeout(() => {
    // 에러 발생
    console.log("doing something wrong");
  }, 2000);
} catch (err) {
  console.log("에러 발생", err);
}
```

또한 에러 처리(추후 학습)를 정상적으로 할 수도 없다.

(당장은 에러가 발생하지 않기 때문)



Promise

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    console.log("Hello Elice");  
  }, 1000);  
});
```

Promise를 이용하면 앞선 문제를 모두 해결하며
비동기 처리를 진행할 수 있다.



Promise

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    console.log("Hello Elice");  
  }, 1000);  
});
```

`new Promise(콜백 함수)`와 같이 new 키워드를 통해 Promise 객체를 생성한다.

이때 콜백 함수가 무조건 인수로 넘겨져야 한다.



Promise – resolve, reject

```
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {  
    console.log("Hello Elice");  
  }, 1000);  
});
```

콜백 함수에는 `resolve`와 `reject` 함수가 인수로 전달된다.



Promise – resolve, reject

```
const promise = new Promise((resolve, reject) => {  
  const fetchedData = fetchData();  
  
  if (fetchedData >= 5) resolve(fetchedData);  
  else reject("fetch failed");  
});
```

`resolve`: 비동기 처리의 성공을 나타냄

`reject`: 비동기 처리의 실패를 나타냄



Promise의 상태

상태	설명
pending	비동기 처리가 완료되기 전, 대기
fulfilled	비동기 처리가 완료되었으며, 성공
rejected	비동기 처리가 완료되었으며, 실패



Promise – resolve, reject

```
const promise = new Promise((resolve, reject) => {  
  const fetchedData = fetchData();  
  
  if (fetchedData >= 5) resolve(fetchedData);  
  else reject("fetch failed");  
});
```

`resolve`: pending → fulfilled

`reject`: pending → rejected



Promise – resolve, reject

```
> const promise = new Promise((resolve, reject) => {});  
< undefined  
> promise  
< ▶ Promise {<pending>}
```

Promise 객체 생성 직후에는 pending 상태



Promise – resolve, reject

```
> function fetchData() { return 10; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
});

< undefined
> promise
< ▶ Promise {<fulfilled>: 10}
```

resolve가 실행되어 fulfilled인 상태



Promise – resolve, reject

```
> function fetchData() { return 3; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
});

< undefined

✖ ▶ Uncaught (in promise) fetch failed

> promise

< ▶ Promise {<rejected>: 'fetch failed'}
```

reject가 실행되어 rejected인 상태

Promise – then, catch, finally

메서드	설명
<code>.then(r => { ... }, e => { ... })</code>	비동기 성공 및 실패 후 처리
<code>.catch(e => { ... })</code>	비동기 실패 후 처리
<code>.finally(() => { ... })</code>	비동기 성공/실패 여부 관계 없이 이후 처리

Promise 객체는 위 세 가지 메서드를 가지며,
이 메서드를 사용해서 **비동기 처리 완료 이후의 처리를 진행**한다.
각 메서드는 모두 Promise 객체를 반환한다.



Promise – then

```
> function fetchData() { return 10; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
}).then(
  (result) => { console.log(`result is ${result}`); },
  (error) => { console.log(`error occurred by ${error}`); }
);

result is 10
< undefined
```

‘then’ 메서드에는 두 가지 콜백 함수가 인수로 전달된다.

1: 비동기 성공 시 처리 (인수: resolve 함수 호출 시 넘긴 값)

2: 비동기 실패 시 처리 (인수: reject 함수 호출 시 넘긴 값)



Promise – catch

```
> function fetchData() { return 3; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
}).catch(
  (error) => { console.log(`error occurred by ${error}`); }
);

error occurred by fetch failed
< undefined
```

‘catch’ 메서드에는 비동기 실패 시 처리를 위한 콜백 함수가 인수로 전달된다.

(인수: reject 함수 호출 시 넘긴 값)



Promise – finally

```
> function fetchData() { return 3; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
}).finally(
  () => { console.log("Promise finished"); }
);
```

Promise finished

< undefined

✖ Uncaught (in promise) fetch failed

‘finally’ 메서드에는 비동기 성공/실패 여부와 관계없이 실행될 콜백 함수가 인수로 전달된다.



Promise Chaining

```
> function fetchData() { return 10; }

const promise = new Promise((resolve, reject) => {
  const fetchedData = fetchData();

  if (fetchedData >= 5) resolve(fetchedData);
  else reject("fetch failed")
}).then(
  (result) => { console.log(`result is ${result}`); }
).catch(
  (error) => { console.log(`error occurred by ${error}`); }
).finally(
  () => { console.log("Promise finished"); }
);

result is 10
Promise finished
< undefined
```

보통 `then`, `catch`, `finally`는 메서드 체이닝*을 통해 같이 사용된다.
(각 메서드는 Promise를 반환하기 때문에 체이닝이 가능하다.)



Promise.resolve

```
const resolvedPromise = Promise.resolve("Hello Elice");  
resolvedPromise.then((result) => { console.log(result); }); // 'Hello Elice'
```

‘Promise.resolve’ 메서드를 사용하면
인수로 전달받은 값을 resolve하는 프로미스를 반환 받을 수 있다.



Promise.reject

```
const rejectedPromise = Promise.reject("Elice Rejected");  
rejectedPromise.catch((error) => { console.log(`error: ${error}`); }); // 'error: Elice Rejected'
```

‘Promise.reject’ 메서드를 사용하면
인수로 전달받은 값을 reject하는 프로미스를 반환 받을 수 있다.



```
// 비동기 작업을 시뮬레이션하는 함수
function fetchData(url, delay) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(`Data from ${url}`);
    }, delay);
  });
}
```

```
// Promise.all을 사용하여 모든 비동기 작업이 완료될 때까지 기다림
Promise.all([
  fetchData('https://api.example.com/data1', 1000),
  fetchData('https://api.example.com/data2', 2000),
  fetchData('https://api.example.com/data3', 1500)
])
  .then((results) => {
    console.log('All promises resolved:');
    results.forEach((result, index) => {
      console.log(`Result ${index + 1}: ${result}`);
    });
  })
  .catch((error) => {
    console.error('One of the promises failed:', error);
  });
```

‘Promise.all’ 메서드를 사용하면
여러 Promise를 병렬적으로 처리할 수 있다.
(인수로 여러 Promise 객체가 담긴 배열 전달)

```
// 비동기 작업을 시뮬레이션하는 함수
function fetchData(url, delay) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(`Data from ${url}`);
    }, delay);
  });
}
```

```
// Promise.all을 사용하여 모든 비동기 작업이 완료될 때까지 기다림
Promise.all([
  fetchData('https://api.example.com/data1', 1000),
  fetchData('https://api.example.com/data2', 2000),
  fetchData('https://api.example.com/data3', 1500)
])
  .then((results) => {
    console.log('All promises resolved:');
    results.forEach((result, index) => {
      console.log(`Result ${index + 1}: ${result}`);
    });
  })
  .catch((error) => {
    console.error('One of the promises failed:', error);
  });
```

인수로 전달된 모든 Promise의 처리가 완료된 후에 then으로 넘어간다.

이때 각 Promise의 결과는 배열의 형태로 전달된다.

마이크로태스크 큐 (Microtask Queue)

```
console.log("1. 스크립트 시작");

setTimeout(() => {
  console.log("4. setTimeout 콜백 실행");
}, 0);

new Promise((resolve) => {
  resolve("3. Promise 콜백 실행");
}).then(result => { console.log(result); })

console.log("2. 스크립트 종료");
```

위 코드는 어떤 순서로 실행될까?

마이크로태스크 큐 (Microtask Queue)

```
console.log("1. 스크립트 시작");

setTimeout(() => {
  console.log("4. setTimeout 콜백 실행");
}, 0);

new Promise((resolve) => {
  resolve("3. Promise 콜백 실행");
}).then(result => { console.log(result); })

console.log("2. 스크립트 종료");
```

앞서 배운 비동기 동작 원리를 고려하면

1 → 2 → 4 → 3의 순서로 예상해 볼 수 있다.

마이크로태스크 큐 (Microtask Queue)

```
> console.log("1. 스크립트 시작");

setTimeout(() => {
  console.log("4. setTimeout 콜백 실행");
}, 0);

new Promise((resolve) => {
  resolve("3. Promise 콜백 실행");
}).then(result => { console.log(result); })

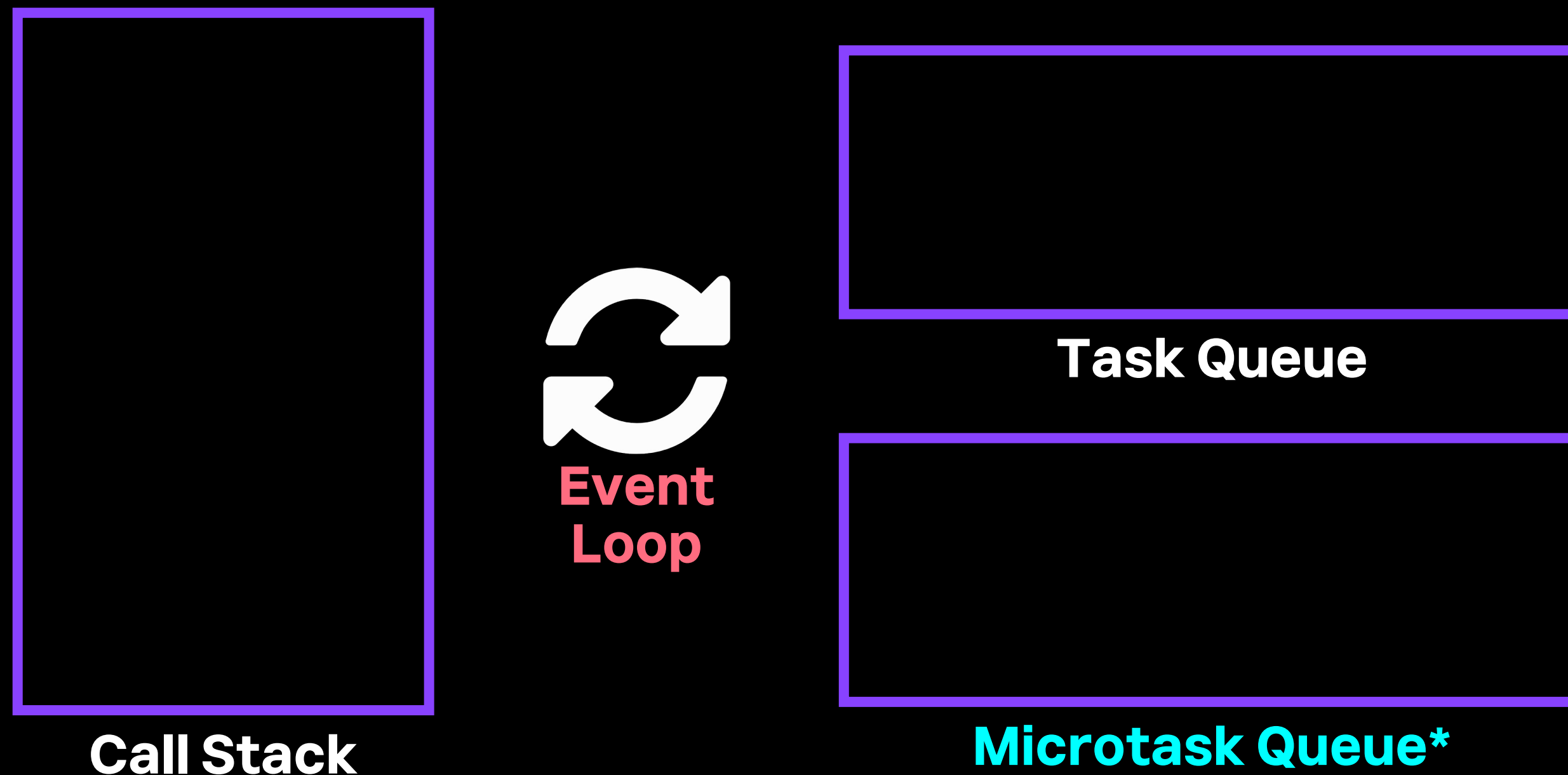
console.log("2. 스크립트 종료");
```

1. 스크립트 시작
2. 스크립트 종료
3. Promise 콜백 실행
< undefined
4. setTimeout 콜백 실행

그러나 실제로는 1 → 2 → 3 → 4의 순서로 실행된다.

왜 이런 것일까?

마이크로태스크 큐 (Microtask Queue)

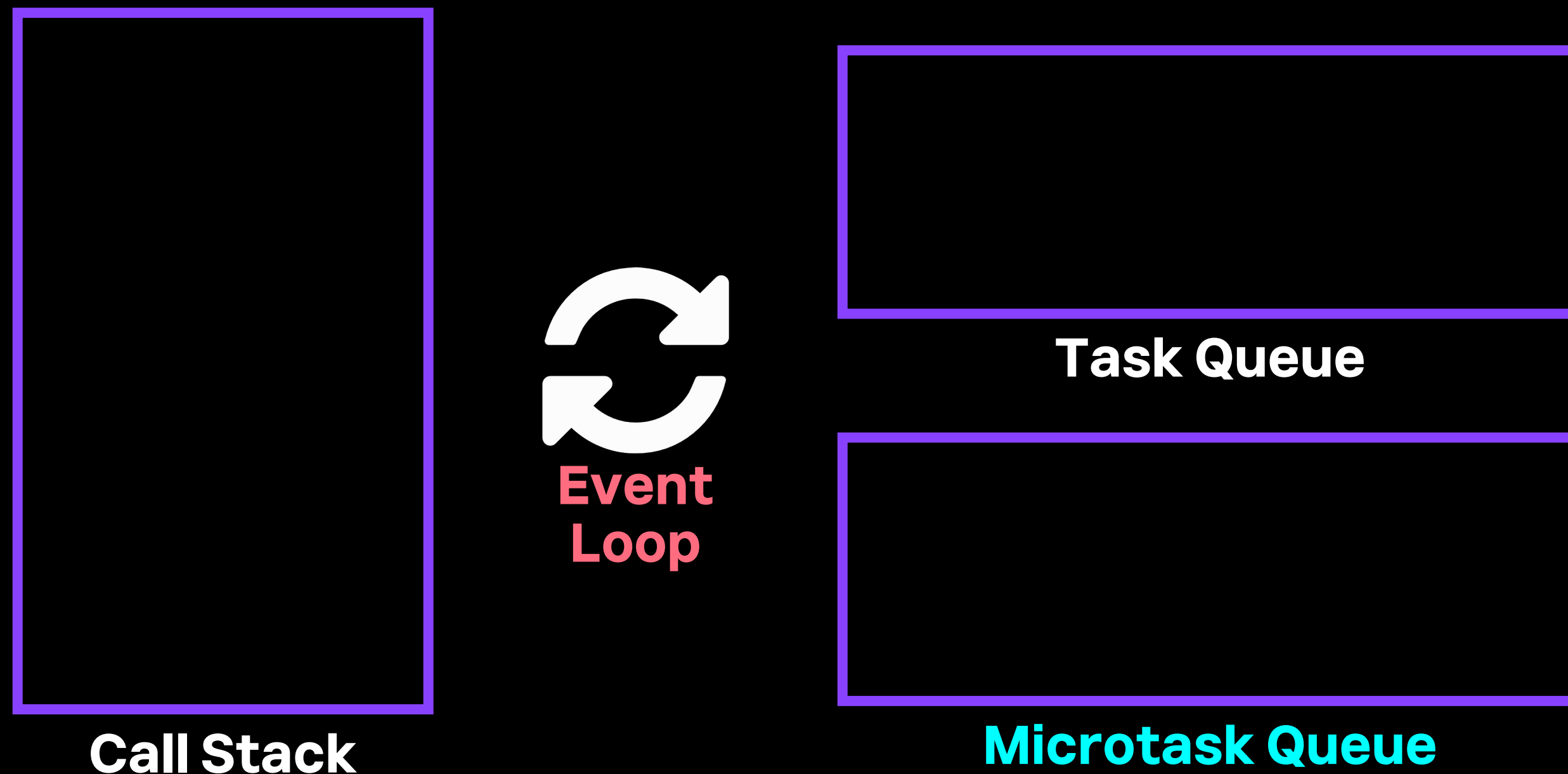


브라우저에는 **마이크로태스크 큐**라는 공간이 하나 더 존재한다.

마이크로태스크 큐는 태스크 큐보다 **우선순위가 높다!**

*Microtask Queue는 Job Queue라고도 한다.

마이크로태스크 큐 (Microtask Queue)



즉, 마이크로태스크 큐에서 대기중인 작업이 먼저 콜 스택으로 들어가고,
마이크로태스크 큐가 비면 태스크 큐의 작업이 콜 스택으로 들어간다.

마이크로태스크 큐 (Microtask Queue)

종류	저장하는 비동기 처리 작업
마이크로태스크 큐	Promise, Ajax
태스크 큐	타이머 함수, 이벤트 핸들러 함수

마이크로태스크 큐 (Microtask Queue)

```
console.log("1. 스크립트 시작");

setTimeout(() => {
  console.log("4. setTimeout 콜백 실행");
}, 0);

new Promise((resolve) => {
  resolve("3. Promise 콜백 실행");
}).then(result => { console.log(result); })

console.log("2. 스크립트 종료");
```

Promise는 마이크로태스크 큐에 저장되므로,
setTimeout의 콜백 함수보다 먼저 실행된다.